

drv_spi.c

```
1  /*
2  * Copyright (c) 2006-2018, RT-Thread Development Team
3  *
4  * SPDX-License-Identifier: Apache-2.0
5  *
6  * Change Logs:
7  * Date           Author       Notes
8  */
9
10 #include <rtthread.h>
11 #include <rtdevice.h>
12
13 #include "drv_spi.h"
14
15 #define SKT_SPI_DEVICE(spi_bus)    (struct skt_spi_bus *) (spi_bus)
16
17 struct skt_spi_bus
18 {
19     struct rt_spi_bus parent;
20     rt_uint32_t spi_periph;
21 };
22
23 static rt_err_t skt_spi_configure(struct rt_spi_device *device,
24                                   struct rt_spi_configuration *configuration)
25 {
26     rt_err_t ret = RT_EOK;
27     struct skt_spi_bus *bus = SKT_SPI_DEVICE(device->bus);
28
29     RT_ASSERT(bus != RT_NULL);
30
31     /* Todo:according to (configuration) set (bus->spi_periph) data_width, prescale, mode. */
32
33     /* Todo:enable SPI (bus->spi_periph) */
34
35     return ret;
36 }
37
38 static rt_uint32_t skt_spi_xfer(struct rt_spi_device *device, struct rt_spi_message *message)
39 {
40     struct skt_spi_bus *bus = SKT_SPI_DEVICE(device->bus);
41     struct rt_spi_configuration *cfg = &device->config;
42     rt_base_t cs_pin = (rt_base_t) device->parent.user_data;
43
44     RT_ASSERT(bus != RT_NULL);
45
46     (void) cs_pin;
47     (void) cfg;
48     /* Todo: transfer message via SPI Hardware(bus->spi_periph) */
49 }
```

```
50     return 0;
51 }
52
53 const static struct rt_spi_ops skt_spi_ops =
54 {
55     skt_spi_configure,
56     skt_spi_xfer
57 };
58
59 int rt_hw_spi_init(void)
60 {
61     rt_err_t ret = RT_EOK;
62     static struct skt_spi_bus spi_bus0;
63
64     /* Todo: Init SPI Hardware */
65
66     ret = rt_spi_bus_register(&spi_bus0.parent, "spi0", &skt_spi_ops);
67
68     return ret;
69 }
70 INIT_DEVICE_EXPORT(rt_hw_spi_init);
71
72
```